GRUPO 09 | TRABAJO APLICATIVO

Detección de fallas en equipos industriales

Comparación y optimización de Árbol de Decisión, Random Forest y Gradient Boosting

Sistemas Inteligentes y Machine Learning

Informe técnico | Clasificación binaria | Python + scikit-learn + Streamlit

Integrantes

COMPLETAR: Integrante 1

COMPLETAR: Integrante 2

COMPLETAR: Integrante 3

10/11/2026

Contenido

1. Resumen ejecutivo
2. Problema, objetivo y dataset
3. Diseño experimental y métrica principal
4. Desarrollo en 16 etapas (01-16)
5. Experimentos: comparación y optimización
6. Evaluación final y análisis de errores
7. Interpretación, estabilidad y limitaciones
8. Respuestas a las preguntas del trabajo
9. Conclusiones y trabajo futuro
10. Reproducibilidad y fuentes

1. Resumen ejecutivo

Se compararon tres algoritmos de árboles (Decision Tree, Random Forest y Gradient Boosting) para predecir falla de máquina (1 = falla, 0 = normal) con **10,000** observaciones, de las cuales solo **3.39%** son fallas. Los tres modelos usaron la misma partición Train/Test (8,000/2,000), los mismos 5 folds, el mismo preprocesamiento sin fuga y una búsqueda aleatoria de 20 configuraciones cada uno.

La métrica principal, fijada antes de ajustar, fue la **PR-AUC** de la clase falla (criterio de selección) con el **recall** como métrica operativa: el umbral de decisión se eligió en validación minimizando el costo esperado (falla no detectada = 10 unidades, falsa alarma = 1). Por validación cruzada ganó **Gradient Boosting** (PR-AUC 0.776 +/- 0.036; segundo: Random Forest, 0.757). La ventaja sobre Random Forest fue de +0.019 de PR-AUC (gana en 3 de 5 folds).

En Test (evaluado una sola vez) el pipeline ganador alcanzó recall **0.809** (IC95% Wilson 0.70-0.88), precisión **0.640** y PR-AUC **0.794**, con 13 fallas no detectadas y 31 falsas alarmas. El modelo se guardó con joblib y se desplegó en una aplicación Streamlit de seis páginas.

Modelo	PR-AUC CV (inicial)	PR-AUC CV (mejor)	Recall CV (mejor)	Tiempo búsqueda (s)
Decision Tree	0.419 +/- 0.054	0.619 +/- 0.028	0.749 +/- 0.043	4
Random Forest	0.730 +/- 0.041	0.757 +/- 0.051	0.863 +/- 0.056	74
Gradient Boosting (ganador)	0.695 +/- 0.033	0.776 +/- 0.036	0.760 +/- 0.030	81

2. Problema, objetivo y dataset

Etapa 01 - Definición del problema

Una planta industrial busca anticipar fallas mecánicas para reducir paradas no programadas y costos de mantenimiento. El problema se formula como **clasificación binaria supervisada**: dadas las condiciones de operación de una máquina (temperaturas, velocidad, torque, desgaste y tipo de producto), predecir si ocurrirá una falla (1) o no (0).

Utilidad de la predicción. Una alerta temprana permite programar una inspección en una ventana planificada en lugar de sufrir una parada imprevista. Las dos equivocaciones tienen costos distintos: una *falsa alarma* cuesta una inspección innecesaria y una *falla no detectada* cuesta una parada no programada. Se asumió una razón de costos 10:1 (supuesto ilustrativo del grupo; en planta debe reemplazarse por costos reales) y se verificó la sensibilidad con razones 5, 10, 20 y 50.

Pregunta de investigación: ¿cuál de los tres algoritmos ofrece la mejor capacidad de generalización para predecir falla de máquina tras ajustar sus hiperparámetros con validación cruzada?

Ficha del dataset

Campo	Valor
Nombre	AI4I 2020 Predictive Maintenance Dataset
Autor	Stephan Matzka
URL Kaggle	https://www.kaggle.com/datasets/stephanmatzka/predictive-maintenance-dataset-ai4i-2020
URL UCI (fuente original)	https://archive.ics.uci.edu/dataset/601/ai4i
DOI	10.24432/C5HS5C
Licencia	CC BY 4.0 (según UCI). VERIFICAR la licencia que muestra la página de Kaggle el día del acceso.
Fecha de acceso	09/11/2026 (archivo entregado por el grupo; confirmar la fecha real de descarga)
Observaciones | columnas	10,000 | 14
Archivo usado	data/ai4i2020.csv
SHA-256 del archivo	dc6630cd9b1f0f853922fad78a1b6436...
Variable objetivo	Machine failure: 1 si ocurre al menos uno de los cinco modos de falla (TWF, HDF, PWF, OSF, RNF); 0 si opera normal.

El dataset original es **sintético**: Matzka (2020) lo diseñó para reflejar el mantenimiento predictivo industrial porque los datos reales son difíciles de obtener y publicar. Esto implica que las fallas siguen reglas explícitas (desgaste de herramienta, disipación de calor, potencia fuera de rango, sobrecarga y fallas aleatorias) y que el desempeño no se puede extrapolar sin más a una planta real (sección 7).

3. Diseño experimental y métrica principal

Elemento	Decisión
Modelos	DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier (scikit-learn)
Partición	Train 8,000 / Test 2,000 (20%), estratificada, semilla 42
Validación cruzada	StratifiedKFold(5, shuffle=True, random_state=42); los MISMOS folds para los tres modelos
Búsqueda	RandomizedSearchCV con 20 configuraciones aleatorias por modelo, dentro de Train
Desbalance	sample_weight 'balanced' idéntico para los tres modelos (los tres se entrenan con la misma ponderación)
Métrica principal	PR-AUC (average precision) de la clase falla, fijada ANTES del ajuste
Métricas secundarias	recall, precisión, F2, ROC-AUC, accuracy, matriz de confusión
Umbral	Elegido en validación (predicciones fuera de muestra de Train) por mínimo costo esperado; nunca en Test
Versiones	python 3.12.3, sklearn 1.9.1, pandas 3.0.6, numpy 2.5.4, scipy 1.18.1, joblib 1.6.0

Justificación de la métrica principal

Con solo 3.39% de fallas, la accuracy es engañosa (un modelo que nunca alerta tendría 96.6% de accuracy). El enunciado pide priorizar recall y PR-AUC. Se adoptó la **PR-AUC como criterio de selección** porque (i) se centra en la clase rara, (ii) no depende de un umbral arbitrario, de modo que compara modelos en todos sus puntos de operación, y (iii) premia modelos que alcanzan recall alto *sin* inundar de falsas alarmas. El **recall** se prioriza en la decisión operativa: el umbral se elige minimizando FN x 10 + FP x 1, lo que empuja el recall hacia arriba cuando una falla no detectada es mucho más cara. Seleccionar por recall con umbral 0.5 habría comparado modelos en puntos de operación distintos y permite 'ganar' alertando casi siempre; por eso se reporta como análisis de sensibilidad en la sección 5.

4. Desarrollo en 16 etapas

Cada subsección resume la evidencia mínima solicitada; el código completo está en train.py (etapas 02-16) y los gráficos en la carpeta figures/.

Etapa 02 - Carga

El CSV se descargó de Kaggle y se importó con `pandas.read_csv`: 10,000 filas y 14 columnas.

Columna	Tipo
UDI	int64
Product ID	str
Type	str
Air temperature [K]	float64
Process temperature [K]	float64
Rotational speed [rpm]	int64
Torque [Nm]	float64
Tool wear [min]	int64
Machine failure	int64
TWF	int64
HDF	int64
PWF	int64
OSF	int64
RNF	int64

Etapa 03 - Exploración

Faltantes: **0**. Filas duplicadas: **0**. Clases: 9661 normales y 339 fallas (3.39%). Tasa de falla por tipo de producto: H 2.09%, L 3.92%, M 2.77%.

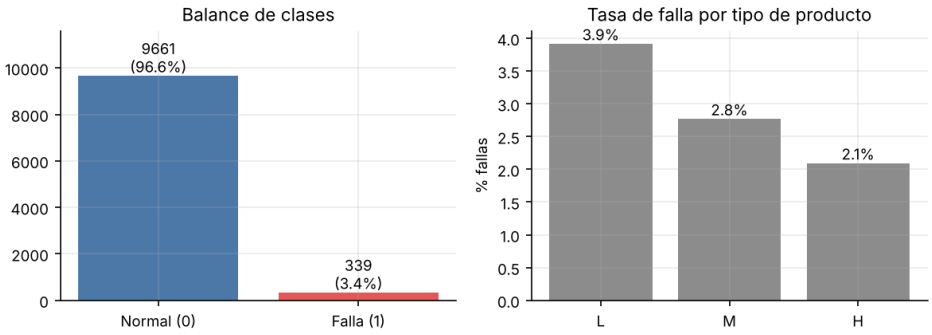


Figura 1. Balance de clases y tasa de falla por tipo de producto.

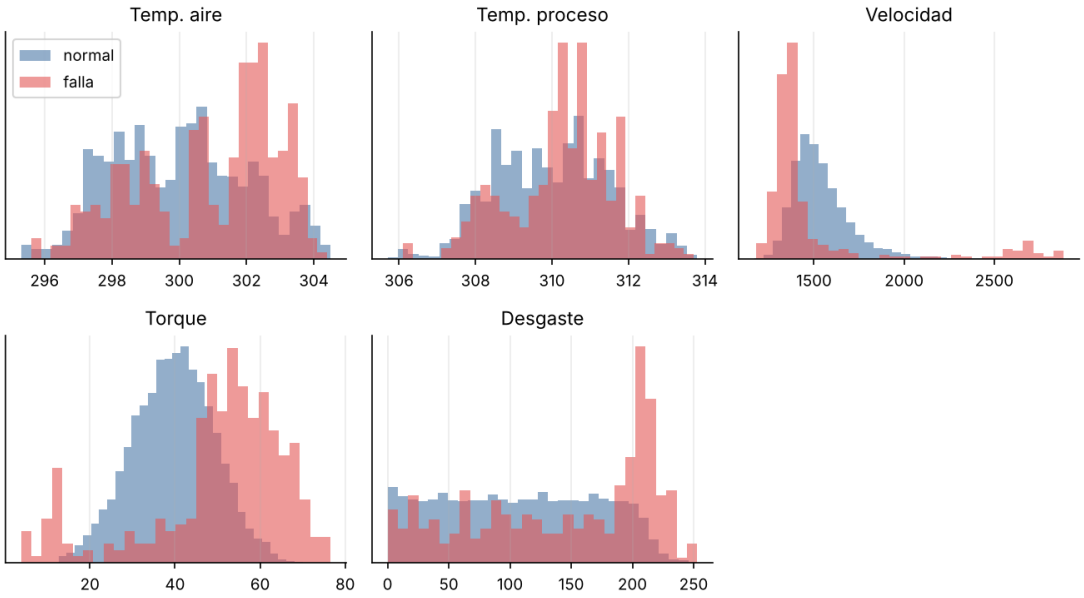


Figura 2. Distribución de cada variable según la clase (densidad). Las fallas se concentran en los extremos.

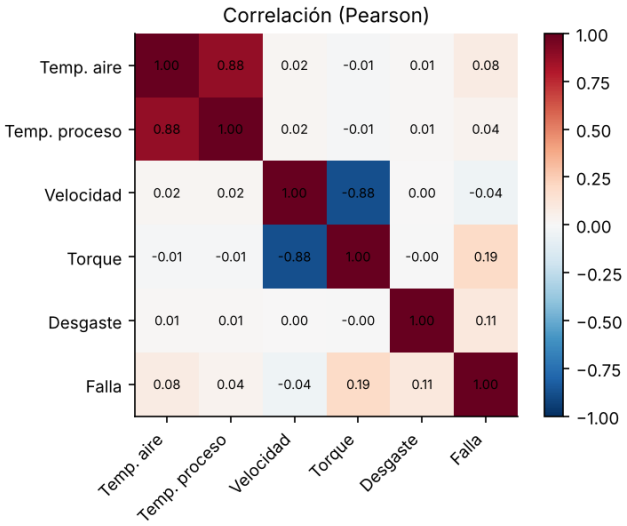


Figura 3. Correlaciones de Pearson. Velocidad y torque: r = -0.88 (potencia casi constante).

Correlaciones lineales con el objetivo: Temp. aire +0.08, Temp. proceso +0.04, Velocidad -0.04, Torque +0.19, Desgaste +0.10. Son débiles porque las fallas dependen de **relaciones no lineales y de interacciones** (por ejemplo, potencia = torque x velocidad, o desgaste x torque); esto justifica usar modelos basados en árboles.

Etapa 04 - Limpieza inicial y revisión de fuga

• **Fuga de información:** Machine failure es el O lógico de TWF, HDF, PWF, OSF y RNF salvo en 27 filas (9 con falla pero sin modo registrado y 18 con algún modo activo pero sin falla; inconsistencias de etiquetado del propio dataset que se conservan como ruido de etiqueta). Esas columnas describen la causa de la falla y no existirían antes de que ocurra; se **excluyen** de X. Cuentas: TWF=46, HDF=115, PWF=95, OSF=98, RNF=19.

• **Identificadores:** UDI (índice 1..n) y Product ID (único por fila: True) se excluyen; no tienen significado físico y permitirían memorizar filas.

• **Type** coincide con la primera letra de Product ID (True), por lo que se conserva solo Type.

• **Duplicados:** 0 filas completas repetidas; 0 filas con predictores idénticos (se conservan: son mediciones distintas).

• **Grupos/tiempo:** no hay identificador de máquina repetido, por lo que no existe fuga entre registros de una misma máquina. Las temperaturas se generaron como caminata aleatoria (autocorrelación temporal); se documenta como limitación.

Etapa 05 - Separar X e y

Variable	Rol	Disponible al predecir?
air_temp (K)	predictor	Sí: sensor ambiental
process_temp (K)	predictor	Sí: sensor del proceso
rpm	predictor	Sí: tacómetro
torque (Nm)	predictor	Sí: sensor de torque
tool_wear (min)	predictor	Sí: contador de uso de herramienta
type (L/M/H)	predictor	Sí: orden de producción
failure	objetivo (y)	No (es lo que se predice)

Etapa 06 - División Train/Test

División estratificada 80/20 con semilla 42: Train 8,000 filas (271 fallas, 3.39%) y Test 2,000 filas (68 fallas, 3.40%). La estratificación conserva la proporción de fallas. Test se reservó y no se usó para elegir modelos, hiperparámetros ni umbral. No se aplicó partición temporal ni por grupos porque el dataset no tiene máquinas repetidas.

Etapa 07 - Preprocesamiento

- **Imputación:** mediana para numéricas y moda para Type. El dataset no tiene faltantes, pero la imputación queda dentro del Pipeline para que el sistema acepte entradas incompletas en producción; sus estadísticos se aprenden solo con Train.
- **Tratamiento numérico:** no se escalan las variables porque los árboles son invariantes a transformaciones monótonas; omitir el escalado mantiene las unidades físicas interpretables.
- **Codificación categórica:** OneHotEncoder con categorías fijas [L, M, H] y handle_unknown='ignore' (no hay orden natural; evita imponer una escala falsa).
- **Sin fuga:** todo se ajusta con fit solo sobre Train (y dentro de cada fold durante la CV) porque el preprocesamiento vive dentro del Pipeline.

Etapa 08 - Construir Pipeline

```
Pipeline([
  ("pre", ColumnTransformer([
      ("num", Pipeline([("imp", SimpleImputer(strategy="median"))]), [air_temp, process_temp, rpm, torque, tool_wear]),
      ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                        ("oh",  OneHotEncoder(handle_unknown="ignore", categories=[["L","M","H"]]))]), [type])])),
  ("model", <DecisionTree | RandomForest | GradientBoosting>)])
```

Se construyó un Pipeline por algoritmo con idéntico preprocesamiento. Es el único objeto que se guarda y despliega.

Etapa 09 - Validación cruzada base

Tres configuraciones iniciales (valores por defecto de scikit-learn; Random Forest y Gradient Boosting con 100 árboles) se evaluaron en los mismos 5 folds de Train.

Modelo	PR-AUC	Recall	Precisión	F2	ROC-AUC
Decision Tree	0.419 +/- 0.054	0.579 +/- 0.065	0.697 +/- 0.055	0.598 +/- 0.057	0.785 +/- 0.032
Random Forest	0.730 +/- 0.041	0.635 +/- 0.031	0.713 +/- 0.049	0.649 +/- 0.031	0.965 +/- 0.012
Gradient Boosting	0.695 +/- 0.033	0.856 +/- 0.025	0.350 +/- 0.018	0.664 +/- 0.023	0.970 +/- 0.008

Etapa 10 - Comparación inicial

Modelo	Árboles	Nodos totales	Profundidad máx.	Tiempo de ajuste/fold (s)	PR-AUC train	PR-AUC CV
Decision Tree	1	423	16	0.03	1.000	0.419

Modelo	Árboles	Nodos totales	Profundidad máx.	Tiempo de ajuste/fold (s)	PR-AUC train	PR-AUC CV
Random Forest	100	46620	26	0.34	1.000	0.730
Gradient Boosting	100	1474	3	0.58	0.881	0.695

Con la configuración inicial el mejor modelo por PR-AUC es **Random Forest**. Los modelos sin límite de profundidad alcanzan PR-AUC de entrenamiento muy alto frente a la de validación: indicio de sobreajuste que la optimización debe controlar. El árbol único es el más simple y rápido; los ensambles cuestan más en tiempo y tamaño.

Etapa 11 - Optimización de hiperparámetros

Se usó **RandomizedSearchCV** dentro de Train con 20 configuraciones por modelo, los mismos folds y la misma métrica (refit sobre PR-AUC). Espacios explorados (los óptimos son relativos a estos espacios):

Modelo	Espacio de búsqueda
Decision Tree	max_depth: [3, 4, 5, 6, 8, 10, 15, None]; min_samples_split: randint(2, 40); min_samples_leaf: randint(1, 25); max_features: [None, 'sqrt', 0.7]; ccp_alpha: uniform(0, 0.01)
Random Forest	n_estimators: randint(100, 251); max_depth: [6, 8, 10, 15, None]; min_samples_split: randint(2, 30); min_samples_leaf: randint(1, 15); max_features: ['sqrt', 'log2', 0.7]; bootstrap: [True, False]
Gradient Boosting	n_estimators: randint(60, 261); learning_rate: uniform(0.02, 0.28); max_depth: randint(2, 5); min_samples_split: randint(2, 30); min_samples_leaf: randint(1, 15); subsample: uniform(0.6, 0.4)

Modelo	best_params_	best_score_ (PR-AUC CV)	Desv. CV	Tiempo (s)
Decision Tree	ccp_alpha=0.0003, max_depth=10, max_features=None, min_samples_leaf=4, min_samples_split=5	0.619	0.028	4
Random Forest	bootstrap=True, max_depth=10, max_features=0.7, min_samples_leaf=5, min_samples_split=10, n_estimators=117	0.757	0.051	74
Gradient Boosting	learning_rate=0.1985, max_depth=4, min_samples_leaf=3, min_samples_split=2, n_estimators=160, subsample=0.9887	0.776	0.036	81

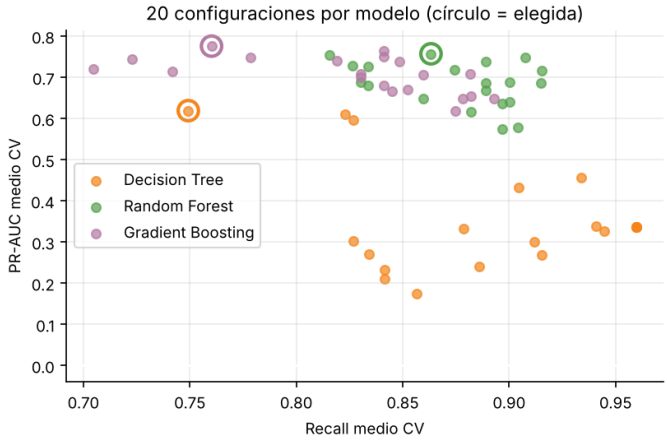


Figura 4. Las 20 configuraciones evaluadas por modelo (recall vs PR-AUC en CV). El círculo marca la elegida.

Etapa 12 - Reentrenar ganador

La selección se hizo exclusivamente con la métrica de validación cruzada: ranking por PR-AUC CV = Gradient Boosting > Random Forest > Decision Tree. **Gradient Boosting** se reentrenó con la mejor configuración sobre todo Train (refit de RandomizedSearchCV). Estabilidad de la ventaja frente a Random Forest (diferencias pareadas por fold): PR-AUC +0.019 (desv. 0.044, gana 3/5 folds); recall -0.103 (desv. 0.064, gana 0/5 folds). Con solo 5 folds correlacionados, estas cifras son indicativas y no una prueba estadística.

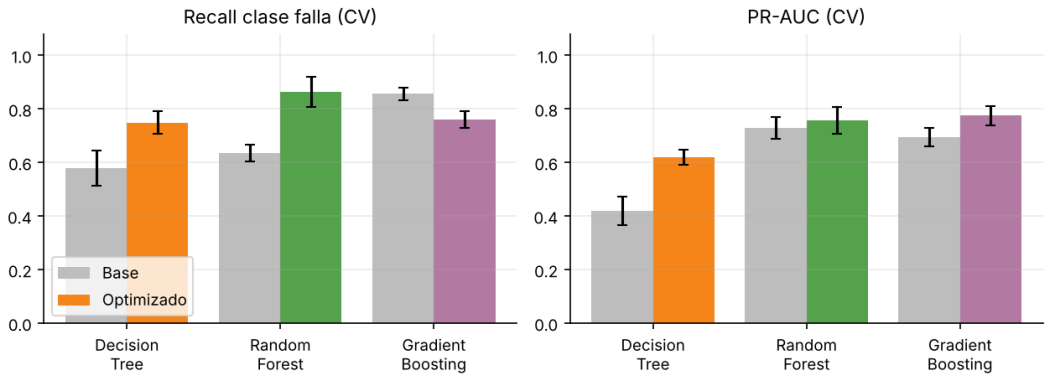


Figura 5. Validación cruzada: configuración base vs optimizada (media y desviación estándar entre folds).

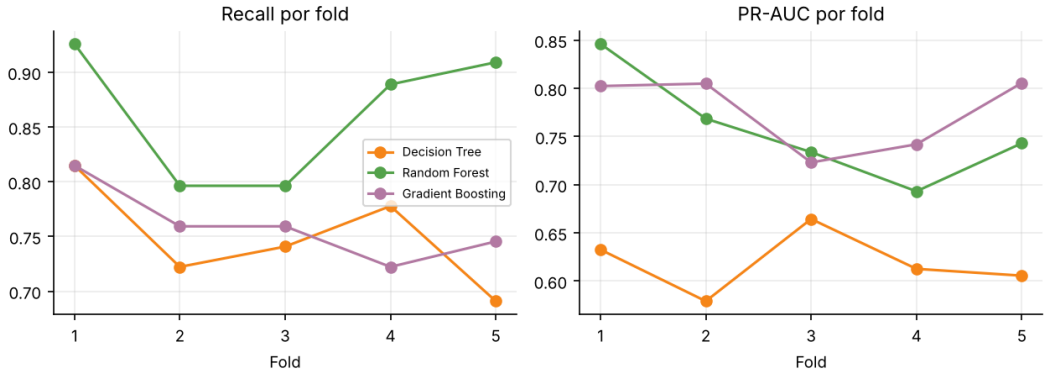


Figura 6. Desempeño fold por fold de la mejor configuración de cada modelo.

Umbral de decisión. Con predicciones fuera de muestra (OOF) del Train se calculó el costo FN x 10 + FP x 1 para umbrales de 0.02 a 0.98 y se tomó el centro de la meseta de costo mínimo: **0.39** (costo OOF 716, frente a 748 con umbral 0.5, 2710 sin modelo y 7729 alertando siempre).

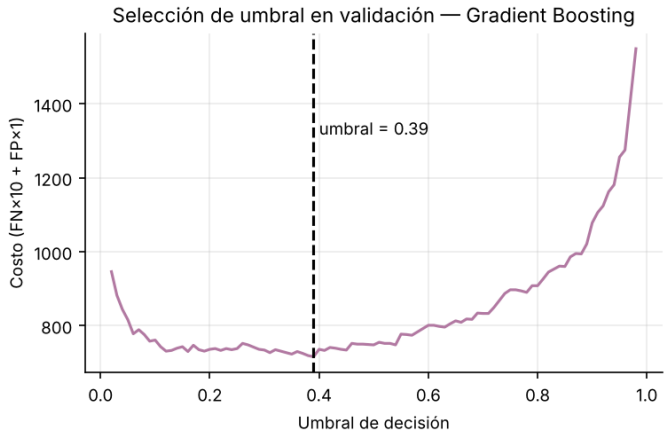


Figura 7. Costo en validación según el umbral; la línea marca el umbral elegido (sin mirar Test).

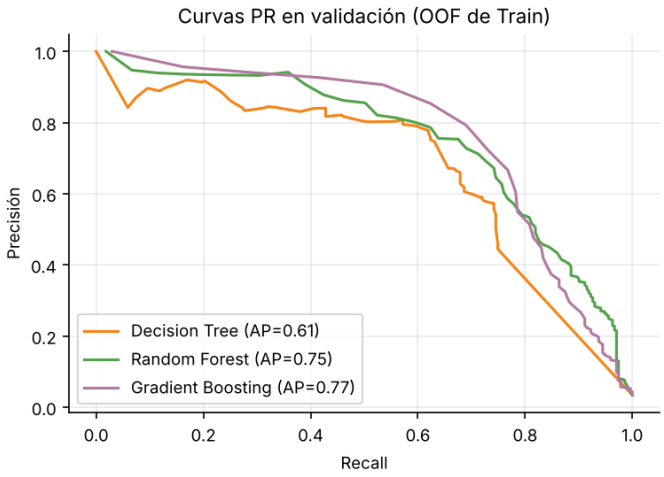


Figura 8. Curvas precisión-recall en validación (predicciones fuera de muestra de Train).

Sensibilidad al criterio de selección. Si en lugar de PR-AUC se hubiera seleccionado por recall CV con precisión >= 0.25, el ganador sería **Decision Tree**:

Modelo	Recall CV	Precisión CV	PR-AUC CV	Misma config. que la elegida?
Decision Tree	0.960	0.264	0.337	no
Random Forest	0.915	0.314	0.716	no
Gradient Boosting	0.893	0.311	0.649	no

Etapa 13 - Evaluación final

El Pipeline ganador se evaluó **una sola vez** en Test con el umbral 0.39 fijado en validación.

Métrica	Valor (umbral 0.39)	Valor (umbral 0.5)
Recall	0.809	0.794
Precisión	0.640	0.675
F2	0.768	0.767
F1	0.714	0.730
Accuracy	0.978	0.980
ROC-AUC	0.963	0.963
PR-AUC	0.794	0.794
FN | FP | TP | TN	13 | 31 | 55 | 1901	14 | 26 | 54 | 1906
Costo (FN x 10 + FP x 1)	161	166

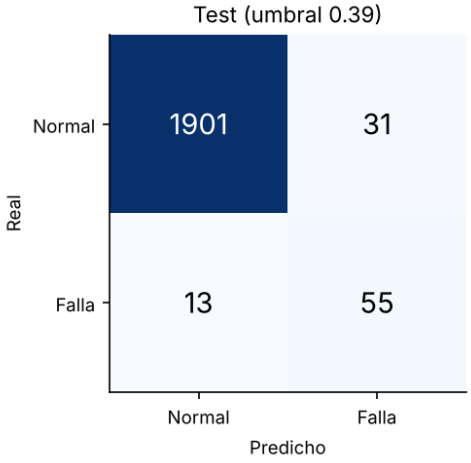


Figura 9. Matriz de confusión en Test.

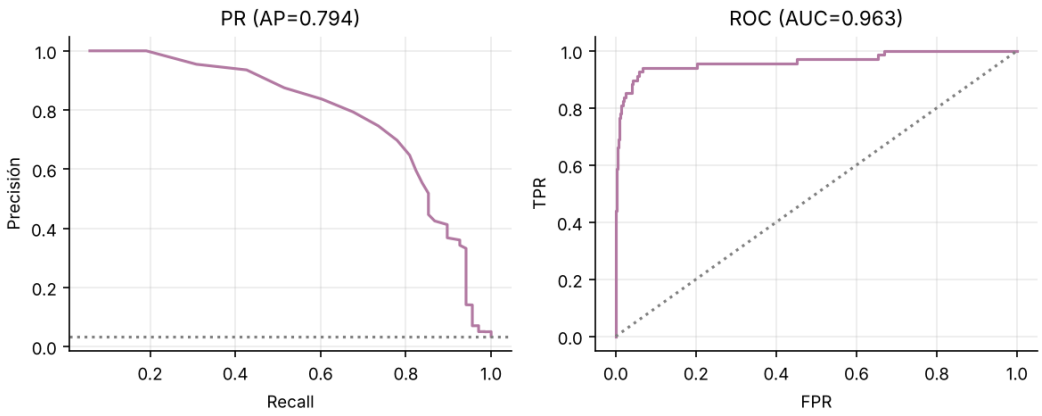


Figura 10. Curvas PR y ROC en Test.

Costo frente a la razón FN:FP (umbral fijo; costo de no usar modelo y de alertar siempre como referencias):

Razón FN:FP	Costo con modelo	Sin modelo (nunca alertar)	Alertar siempre
5:1	96	340	1932
10:1	161	680	1932
20:1	291	1360	1932
50:1	681	3400	1932

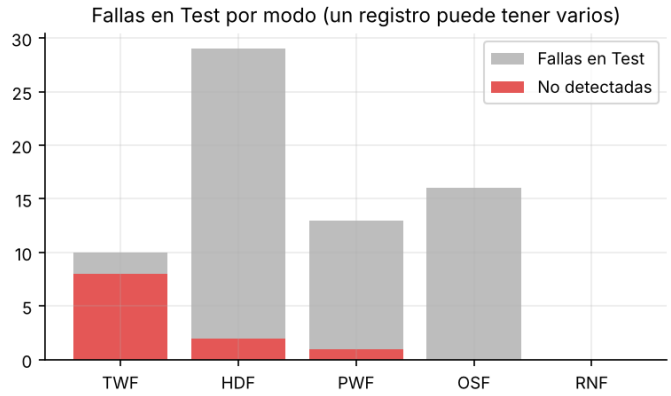


Figura 11. Fallas de Test por modo y cuántas no se detectaron (análisis post-hoc; los modos no son predictores).

Errores por modo (post-hoc): TWF: 8 de 10 no detectadas; HDF: 2 de 29 no detectadas; PWF: 1 de 13 no detectadas; OSF: 0 de 16 no detectadas; RNF: 0 de 0 no detectadas. En total hubo 13 fallas no detectadas y 31 falsas alarmas. Ejemplos de fallas no detectadas:

air_temp	process_temp	rpm	torque	tool_wear	type
303.7	312.1	1363	51.8	90	L
300.8	312.4	1465	59.1	91	L
297.8	308.9	1871	25.6	200	L
299.7	309.3	1399	41.9	221	M
302.2	311.3	1530	37.3	207	L
300.6	310.3	1648	30.5	217	L

Etapa 14 - Interpretación

Los detalles están en la sección 7 (importancia de variables, sobreajuste, estabilidad y limitaciones).

Etapa 15 - Predicción con cinco registros nuevos

Cinco configuraciones de operación plausibles y completamente descritas (valores dentro de los rangos del dataset). La alerta se emite si P(falla) >= umbral, vigilancia si P >= umbral/2.

ID	Descripción	Aire K	Proc. K	rpm	Torque	Desg.	Tipo	Potencia W	Dif. T	P(falla)	Estado
R1	Operación nominal, producto M, herramienta casi nueva	299.8	309.9	1540	40.0	15	M	6451	10.1	0.1%	NORMAL
R2	Herramienta muy desgastada con torque alto (riesgo de sobrecarga), tipo L	300.4	310.6	1420	62.0	215	L	9220	10.2	99.9%	ALERTA
R3	Baja velocidad y poca diferencia de temperatura (mala disipación de calor)	301.5	309.7	1290	52.0	120	M	7025	8.2	0.2%	NORMAL
R4	Torque muy bajo con velocidad alta (potencia fuera de rango)	298.9	308.7	2050	12.0	60	H	2576	9.8	98.2%	ALERTA
R5	Caso intermedio: desgaste y torque moderados-altos, tipo H	300.9	311.0	1480	48.0	170	H	7439	10.1	0.1%	NORMAL

Lectura física esperada según las reglas documentadas del dataset: R2 combina desgaste alto y torque alto (desgaste x torque supera el límite de sobrecarga del tipo L); R3 tiene diferencia de temperatura menor a 8.6 K con velocidad menor a 1380 rpm (disipación de calor deficiente); R4 tiene potencia fuera del rango 3500-9000 W; R1 es operación nominal y R5 es un caso intermedio sin regla activada. El modelo coincide con esa lectura en 4 de 5 casos (R1, R2, R4, R5). Discrepa en R3: el modelo no alertó un caso que la regla física marca como falla (o viceversa), lo que ilustra una limitación real: señales como la diferencia de temperatura o la potencia son combinaciones de varias variables y un árbol sobre variables crudas las aprende de forma imperfecta. Por eso la sección 8 propone agregar variables derivadas.

Etapa 16 - Guardar y desplegar

El Pipeline ganador se guardó con `joblib.dump` en model/pipeline.joblib junto con el umbral y los rangos de entrenamiento. La aplicación Streamlit (app.py) carga el artefacto una sola vez (st.cache_resource) y **no realiza búsqueda de hiperparámetros** en cada consulta. Páginas: (1) problema y dataset, (2) análisis exploratorio, (3) comparación base vs optimizada, (4) parámetros encontrados, (5) evaluación final, (6) formulario predictivo con alertas. Ejecución: `streamlit run app.py`.

5. Experimentos: comparación y optimización

Tabla resumen solicitada por el enunciado (CV inicial vs mejor CV, hiperparámetros óptimos y tiempo):

Modelo	CV inicial (PR-AUC)	Mejor CV (PR-AUC)	Hiperparámetros óptimos	Tiempo (s)
Decision Tree	0.419 +/- 0.054	0.619 +/- 0.028	ccp_alpha=0.0, max_depth=10, max_features=None, min_samples_leaf=4, min_samples_split=5	4
Random Forest	0.730 +/- 0.041	0.757 +/- 0.051	bootstrap=True, max_depth=10, max_features=0.7, min_samples_leaf=5, min_samples_split=10, n_estimators=117	74
Gradient Boosting	0.695 +/- 0.033	0.776 +/- 0.036	learning_rate=0.199, max_depth=4, min_samples_leaf=3, min_samples_split=2, n_estimators=160, subsample=0.989	81

Modelo	Recall CV	Precisión CV	F2 CV	ROC-AUC CV	Nodos totales	Árboles	Predicción (ms/1000 filas)
Decision Tree	0.749 +/- 0.043	0.453	0.659	0.867	149	1	3.6
Random Forest	0.863 +/- 0.056	0.431	0.719	0.972	18149	117	11.1
Gradient Boosting	0.760 +/- 0.030	0.681	0.742	0.966	4442	160	7.1

Por qué ganó cada configuración. Con la métrica PR-AUC y 20 configuraciones por modelo, cada mejor configuración es la de mayor PR-AUC media en CV dentro del espacio explorado:

• **Decision Tree:** ccp_alpha=0.0003, max_depth=10, max_features=None, min_samples_leaf=4, min_samples_split=5. PR-AUC CV 0.619 (inicial 0.419).

• **Random Forest:** bootstrap=True, max_depth=10, max_features=0.7, min_samples_leaf=5, min_samples_split=10, n_estimators=117. PR-AUC CV 0.757 (inicial 0.730).

• **Gradient Boosting:** learning_rate=0.1985, max_depth=4, min_samples_leaf=3, min_samples_split=2, n_estimators=160, subsample=0.9887. PR-AUC CV 0.776 (inicial 0.695).

En general, los hiperparámetros que limitan la complejidad (profundidad máxima, mínimo de muestras por hoja, ccp_alpha, subsample, learning_rate bajo con más árboles) reducen la varianza y mejoran la generalización cuando hay pocas muestras positivas; los que aumentan capacidad (n_estimators, max_depth alto) ayudan hasta cierto punto y luego solo aumentan tiempo. La tabla de arriba y cv_results_*.csv permiten comprobar qué parámetros movieron realmente la métrica.

6. Evaluación final y análisis de errores

El ganador por CV, **Gradient Boosting**, obtuvo en Test: recall 0.809, precisión 0.640, F2 0.768, PR-AUC 0.794 y ROC-AUC 0.963. Como Test contiene solo 68 fallas, el recall tiene una incertidumbre apreciable: IC95% de Wilson = [0.70, 0.88]. Comparación Test vs validación: PR-AUC 0.794 frente a 0.776 en CV (diferencia +0.019); una diferencia pequeña indica que la estimación de CV fue honesta.

Costo operativo. Con la razón 10:1 el costo en Test fue 161, frente a 680 si no se usara modelo y 1932 si se alertara siempre.

¿Qué ocurriría si se eligiera el modelo mirando Test? (análisis post-hoc)

Calculado DESPUÉS de congelar el ganador; sólo ilustra el sesgo de elegir con Test. Se evaluaron los tres modelos optimizados en Test, cada uno con su propio umbral de validación:

Modelo	Recall	Precisión	PR-AUC	FN	FP	Costo
Decision Tree	0.765	0.605	0.687	16	34	194
Random Forest	0.868	0.404	0.814	9	87	177
Gradient Boosting	0.809	0.640	0.794	13	31	161

Ranking por PR-AUC en Test: Random Forest > Gradient Boosting > Decision Tree; ranking por CV: Gradient Boosting > Random Forest > Decision Tree. El primer lugar cambia, lo que muestra cómo Test habría alterado la decisión. En cualquier caso, elegir mirando Test es incorrecto: con 68 fallas el ruido de muestreo es comparable a las diferencias entre modelos, y el máximo de varias estimaciones ruidosas está sesgado hacia arriba (optimismo de selección). Test dejaría de ser una estimación imparcial: habría que reportar el desempeño ya optimizado sobre esos mismos datos. Por eso la decisión se tomó solo con CV y Test se usó una vez.

7. Interpretación, estabilidad y limitaciones

Variables importantes

Importancia por permutación en Train (caída de PR-AUC al barajar una variable): torque 0.792, air_temp 0.492, rpm 0.383, tool_wear 0.348, process_temp 0.282, type 0.009. La variable más influyente es **torque**. La importancia por impureza (derecha) puede sobrevalorar variables continuas, por eso se contrasta con la de permutación.

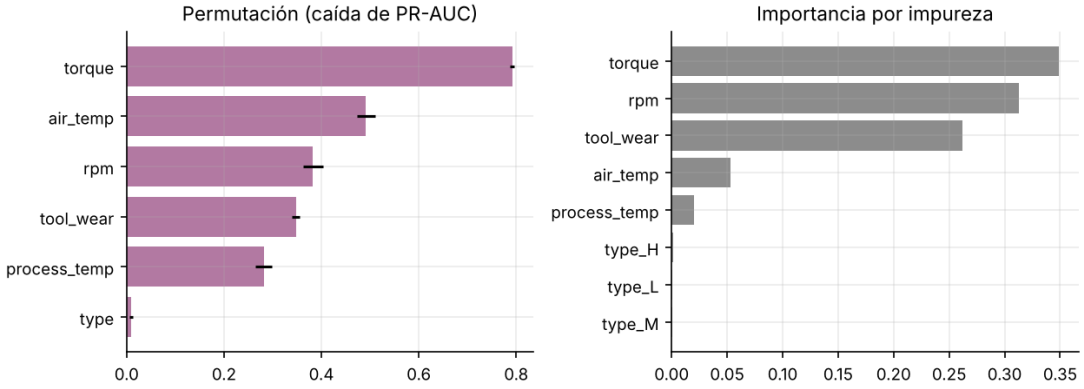


Figura 12. Importancia de variables (permutación en Train y por impureza).

Sobreajuste

Modelo	PR-AUC train	PR-AUC CV	Brecha	Recall train	Recall CV
Decision Tree	0.870	0.619	0.251	1.000	0.749
Random Forest	0.982	0.757	0.225	1.000	0.863
Gradient Boosting	1.000	0.776	0.224	1.000	0.760

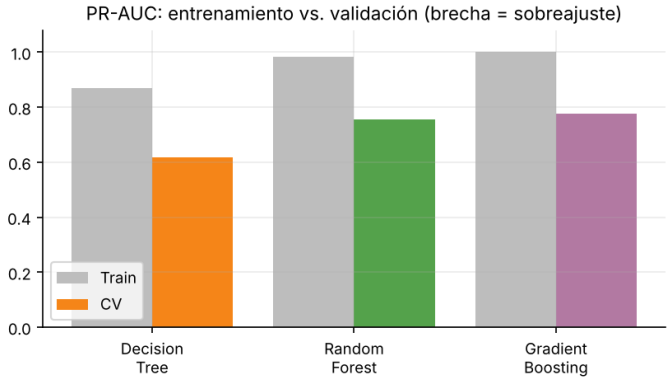


Figura 13. PR-AUC en entrenamiento vs validación (la brecha mide sobreajuste).

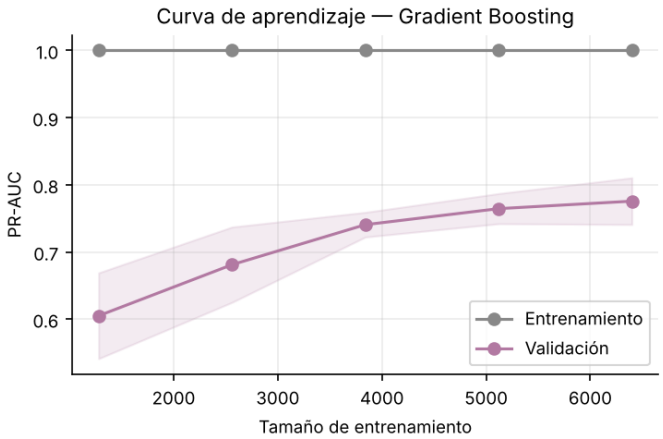


Figura 14. Curva de aprendizaje del ganador: si la validación sigue subiendo, más datos ayudarían.

La PR-AUC de validación pasa de 0.605 (20% del Train) a 0.776 (100%); la curva aún crece, así que más datos probablemente mejorarían el modelo. Brecha train-CV del ganador: 0.224.

Estabilidad

Entre folds, la desviación de la PR-AUC del ganador es 0.036 y la del recall 0.030. La ventaja sobre Random Forest es +0.019 (gana en 3 de 5 folds). Como la desviación entre folds es del mismo orden que algunas diferencias, la conclusión debe leerse como 'ventaja consistente' solo si gana en todos o casi todos los folds.

Limitaciones

- **Datos sintéticos:** las fallas siguen reglas explícitas; los árboles pueden aproximarlas muy bien, algo que no ocurriría con ruido de planta real.
- **Pocas fallas:** 68 positivos en Test generan intervalos de confianza amplios (recall IC95% 0.70-0.88).
- **Autocorrelación temporal:** las temperaturas son caminatas aleatorias; una división aleatoria puede ser algo optimista frente a una validación cronológica.
- **Costos supuestos:** la razón 10:1 es un supuesto; el umbral óptimo cambia con los costos reales.
- **Probabilidades no calibradas:** los modelos de árboles (sobre todo con ponderación de clases) no entregan probabilidades calibradas; se usan para rankear riesgo y alertar con un umbral.
- **Horizonte:** el modelo predice falla en el mismo registro, no con anticipación de horas o días.

8. Respuestas a las preguntas del trabajo

1. ¿Qué configuración de hiperparámetros ganó en cada algoritmo y por qué?

• **Decision Tree:** ccp_alpha=0.0003, max_depth=10, max_features=None, min_samples_leaf=4, min_samples_split=5. Fue la de mayor PR-AUC media en CV (0.619 +/- 0.028) entre 20 configuraciones evaluadas en los mismos folds.

• **Random Forest:** bootstrap=True, max_depth=10, max_features=0.7, min_samples_leaf=5, min_samples_split=10, n_estimators=117. Fue la de mayor PR-AUC media en CV (0.757 +/- 0.051) entre 20 configuraciones evaluadas en los mismos folds.

• **Gradient Boosting:** learning_rate=0.1985, max_depth=4, min_samples_leaf=3, min_samples_split=2, n_estimators=160, subsample=0.9887. Fue la de mayor PR-AUC media en CV (0.776 +/- 0.036) entre 20 configuraciones evaluadas en los mismos folds.

2. ¿Qué algoritmo ganó según validación cruzada? ¿Qué tan estable es esa ventaja?

Ganó **Gradient Boosting** (PR-AUC CV 0.776). Frente a Random Forest: diferencia media +0.019, desviación 0.044, gana en 3 de 5 folds (Figura 6). Con 5 folds correlacionados no se afirma significancia estadística; se informa la consistencia por fold.

3. ¿Qué errores y limitaciones mostró el ganador en el conjunto Test?

13 fallas no detectadas (TWF: 8/10, HDF: 2/29, PWF: 1/13, OSF: 0/16 por modo) y 31 falsas alarmas; precisión 0.640. Limitaciones: pocas fallas en Test (IC95% de recall 0.70-0.88), datos sintéticos y costos supuestos (sección 7).

4. ¿Qué ocurriría si se eligiera el modelo mirando Test?

Test dejaría de ser independiente: la elección se ajustaría al ruido de esos datos y la métrica reportada sería optimista (sesgo de selección). En este trabajo, el ranking por Test fue Random Forest > Gradient Boosting > Decision Tree y por CV Gradient Boosting > Random Forest > Decision Tree (sección 6).

5. ¿Qué cambiaría antes de utilizarlo en un entorno real?

• Validar con datos reales de planta y con partición cronológica o por máquina.

• Estimar los costos reales de falsa alarma y de parada no programada y recalcular el umbral.

• Calibrar probabilidades (Platt/isotónica) y fijar niveles de alerta con operación y mantenimiento.

• Agregar variables físicas derivadas (potencia = torque x velocidad, diferencia de temperatura, desgaste x torque) y evaluar su aporte con la misma CV.

• Monitorear deriva de datos y rendimiento, y reentrenar periódicamente; mantener un esquema con humano en el lazo.

• Definir el horizonte de predicción (falla en las próximas horas) y rediseñar la variable objetivo en consecuencia.

• Pruebas de robustez: valores faltantes, sensores descalibrados, rangos fuera de entrenamiento (la app ya avisa).

9. Conclusiones y trabajo futuro

• Bajo el mismo problema, partición y folds, **Gradient Boosting** fue el mejor por validación cruzada (PR-AUC 0.776) y su desempeño en Test (0.794 PR-AUC; recall 0.809) fue consistente con esa estimación.

• Optimizar hiperparámetros con CV dentro de Train mejora o iguala la configuración inicial y controla el sobreajuste; los resultados óptimos son relativos al espacio explorado.

• La elección del umbral por costo en validación convierte las probabilidades en una regla de alerta alineada con el costo de negocio, sin tocar Test.

• El análisis de sensibilidad muestra que la conclusión depende de la métrica: seleccionar por recall con umbral fijo favorece modelos que alertan en exceso. Fijar la métrica antes del ajuste y justificarla es parte del resultado.

• Trabajo futuro: datos reales, validación temporal, calibración, variables derivadas y comparación con otros modelos.

10. Reproducibilidad y fuentes

Reproducibilidad

```
pip install -r requirements.txt
# 1) colocar el CSV real en data/ai4i2020.csv  (Kaggle/UCI, ver README) y completar dataset_info.json
python run_all.py          # entrena, genera figuras, informe PDF y presentación
streamlit run app.py       # aplicación de predicción
```

Semilla 42; scikit-learn 1.9.1; pandas 3.0.6; numpy 2.5.4; Python 3.12.3. Presupuesto de búsqueda: 20 configuraciones x 5 folds x 3 modelos. Tiempo total de la ejecución: 0.7 min.

Fuentes

- S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," 2020 Third International Conference on Artificial Intelligence for Industries (AI4I), 2020, pp. 69-74, doi: 10.1109/AI4I49448.2020.00023.
- AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C (licencia CC BY 4.0).
- Kaggle: https://www.kaggle.com/datasets/stephanmatzka/predictive-maintenance-dataset-ai4i-2020
- Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830, 2011.
- Bergstra, J. y Bengio, Y. Random Search for Hyper-Parameter Optimization. JMLR 13, 281-305, 2012.
- Saito, T. y Rehmsmeier, M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 2015.
- Guía del trabajo aplicativo Grupo 09, Sistemas Inteligentes y Machine Learning.